HOLLISTER INCORPORATED

Event-Driven Architecture Design Guide

MQTT + Kafka Hybrid Architecture
From Shop Floor to Boardroom

The Architecture Metaphor

MQTT is the ***Nervous System***
Fast, lightweight, responsive to the "here and now."

Kafka is the ***Brain and Memory***
Capable of storing, analyzing, and replaying everything that has happened

Document ID	HOLL-ARCH-EDA-001
Version	1.0
Date	February 2026
Related Documents	HOLL-ARCH-001, HOLL-TECH-PLC-MES-001

Table of Contents

1. Executive Overview

1.1 The Integration Challenge

Hollister's Digital Factory transformation requires seamless data flow from the shop floor (PLCs, sensors, MES) to enterprise systems (ERP, CRM, PLM, Supply Chain). This presents a fundamental architectural challenge: shop floor systems operate in milliseconds with high-frequency, ephemeral data, while enterprise systems operate in minutes or hours with durable, transactional data.

A single messaging technology cannot optimally serve both worlds. Attempting to force-fit one solution leads to either:

- Using MQTT for everything: Loses data durability, replay capability, and enterprise integration patterns
- Using Kafka for everything: Adds unnecessary latency and complexity at the edge, overwhelms brokers with millions of tiny topics

1.2 The Hybrid Solution

This guide defines a Hybrid Event-Driven Architecture (EDA) that uses the right tool for each layer:

Layer	Technology	Why
Edge/Shop Floor	MQTT	Lightweight, handles unreliable networks, millisecond response, millions of topics
Integration Bridge	Kafka Connect / Bridge	Aggregates, filters, transforms, and routes data between worlds
Enterprise	Apache Kafka	Durable storage, replay capability, exactly-once semantics, enterprise consumers

2. MQTT vs. Kafka: Understanding the Differences

While both MQTT and Kafka use "topics" to categorize data, they serve fundamentally different purposes. Understanding these differences is critical to designing an effective hybrid architecture.

2.1 Comprehensive Feature Comparison

Feature	MQTT	Kafka
Primary Role	Real-time messaging for IoT/edge devices	Durable event streaming and log storage
Topic Structure	Hierarchical: factory/line/machine/sensor. Supports wildcards (+, #)	Flat: machine_events. No hierarchy; partitioning handles scaling
Message Persistence	Ephemeral: Gone after delivery (unless Retained flag). QoS ensures delivery, not storage	Durable: Stored on disk for configurable retention (days, months, years). Full replay capability
Delivery Model	Push: Broker immediately pushes to active subscribers	Pull: Consumers request data when ready, at their own pace
Topic Scaling	Scales to millions of topics easily. Each sensor can have its own topic	Optimized for high throughput per topic. Limited total topic count (thousands)
Message Size	Optimized for small messages (bytes to KB)	Handles larger messages efficiently (KB to MB)
Network Tolerance	Excellent: Designed for unreliable, high-latency networks. Small protocol overhead	Good: Requires stable network. Higher protocol overhead
Consumer Groups	Limited: Shared subscriptions (MQTT 5.0) provide basic load balancing	Advanced: Consumer groups with partition assignment, rebalancing, offset management
Ordering Guarantees	Per-topic ordering only	Per-partition ordering with configurable partitioning strategy
Replay Capability	None: Messages are fire-and-forget (Retained gives only last message)	Full: Consumers can rewind to any offset and replay history
Latency	Sub-millisecond to low milliseconds	Low milliseconds to tens of milliseconds
Best For	PLCs, sensors, edge devices, MES real-time, mobile/constrained clients	ERP integration, analytics, audit logs, event sourcing, microservices

2.2 Topic Namespace Isolation

A critical point: MQTT topics and Kafka topics exist in completely separate namespaces. They are not shared natively and must be explicitly bridged.

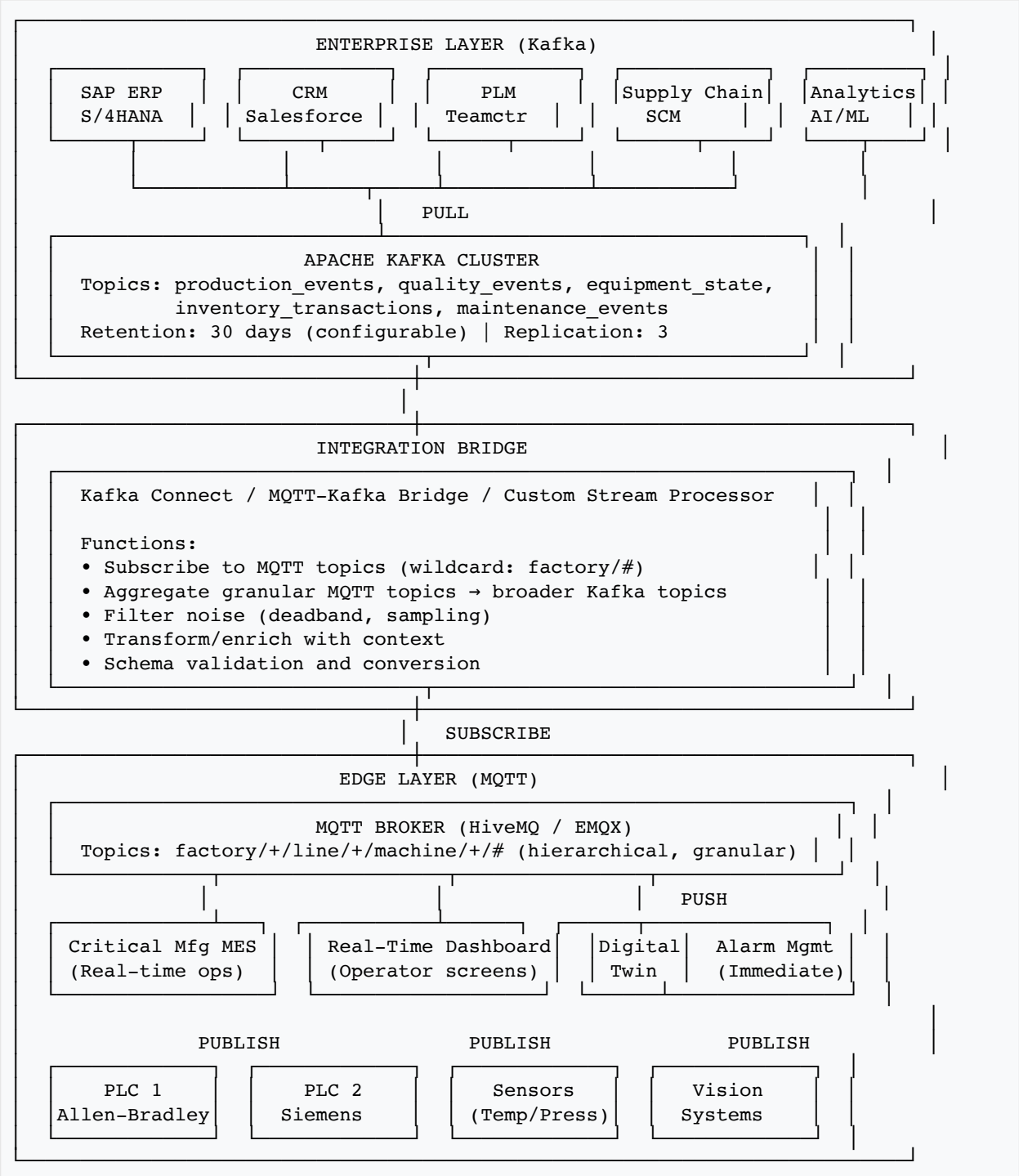
This means:

- An MQTT topic factory1/line3/plc/temp has no relationship to a Kafka topic of the same name
- Data must flow through a bridge component that subscribes to MQTT and publishes to Kafka
- Topic mapping and transformation logic lives in the bridge layer

3. The Hybrid Architecture

3.1 Three-Layer Design

The hybrid architecture consists of three distinct layers, each optimized for its specific requirements:



4. Edge Layer: MQTT for Shop Floor

4.1 Why MQTT at the Edge

PLCs (like Siemens S7, Allen-Bradley ControlLogix) and shop floor sensors operate in resource-constrained, often unreliable network environments. MQTT is purpose-built for this world:

- Lightweight Protocol: Minimal overhead (2-byte header minimum), perfect for constrained devices
- Unreliable Networks: Built-in QoS levels handle packet loss, connection drops gracefully
- Millisecond Response: MES can react to machine state changes in real-time
- Hierarchical Topics: Natural mapping to physical plant structure (site/area/line/machine/sensor)
- Wildcard Subscriptions: MES subscribes to factory/+/+/state to monitor all machines

4.2 MQTT Topic Design for Manufacturing

The MQTT topic hierarchy mirrors the physical plant structure and ISA-95 equipment model:

```
hollister/
├── {site}/                              # BAL, SDR, LIB, KAU, FRE
│   ├── {area}/                          # MH1 (Molding Hall 1), PKG, CLN
│   │   ├── {line}/                      # LINE01, LINE02
│   │   │   ├── {equipment}/            # MOLD01, PRESS02, ROBOT03
│   │   │   │   ├── state                # Equipment state (PackML)
│   │   │   │   ├── telemetry/           # Real-time sensor data
│   │   │   │   │   ├── temperature
│   │   │   │   │   ├── pressure
│   │   │   │   │   ├── vibration
│   │   │   │   │   └── speed
│   │   │   │   ├── production/          # Production events
│   │   │   │   │   ├── cycle_complete
│   │   │   │   │   ├── count
│   │   │   │   │   └── reject
│   │   │   │   ├── quality/             # Quality measurements
│   │   │   │   │   ├── inspection
│   │   │   │   │   └── spc
│   │   │   │   ├── alarms/              # Active alarms
│   │   │   │   │   ├── active
│   │   │   │   │   └── acknowledged
│   │   │   │   └── maintenance/         # Maintenance events
│   │   │   │       ├── request
│   │   │   │       └── complete

Example Subscriptions:
• MES subscribes to: hollister/BAL/+/+/+/state           (all equipment states in
Ballina)
• Digital Twin:      hollister/BAL/MH1/#                 (everything in Molding Hall
1)
• Alarm System:      hollister/+/+/+/+/alarms/active     (all active alarms
```

4.3 MQTT QoS Levels for Manufacturing

QoS	Guarantee	Use Case	Example
0	At most once (fire and forget)	High-frequency telemetry where occasional loss is acceptable	Temperature readings every 100ms

1	At least once (may duplicate)	Production events where loss is unacceptable	Cycle complete, part count
2	Exactly once (guaranteed single delivery)	Critical events requiring audit trail	Quality failures, safety alarms

4.4 Edge Consumers (MQTT Subscribers)

These systems subscribe directly to MQTT for real-time responsiveness:

Consumer	Subscription Pattern	Latency Requirement
MES (Critical Mfg)	hollister/{site}/+/+/+/production/#, .../state	< 100ms for state changes, cycle complete
Digital Twin	hollister/{site}/#	< 500ms for visualization sync
Alarm Management	hollister/+/+/+/+/alarms/#	< 50ms for safety-critical alarms
Operator Dashboards	hollister/{site}/{area}/#	< 1s for screen updates
Edge Analytics	hollister/{site}/+/+/+/telemetry/#	Real-time for SPC, OEE calculation

5. Integration Bridge: Connecting Worlds

5.1 Bridge Purpose and Functions

The Integration Bridge is the critical translation layer between the MQTT edge world and the Kafka enterprise world. It performs several essential functions:

Function	Description
Topic Aggregation	Consolidate many granular MQTT topics into fewer broad Kafka topics. Example: 1000 machine state topics → 1 Kafka equipment_state topic
Filtering	Apply deadband filtering, sampling, and noise reduction. Not every sensor reading needs to reach Kafka—only significant changes
Transformation	Convert MQTT JSON payloads to Avro schemas, enrich with metadata, normalize timestamps to UTC
Batching	Collect multiple MQTT messages into batches for efficient Kafka writes, reducing broker load
Schema Validation	Validate incoming data against registered schemas, reject malformed messages with logging
Partitioning	Determine Kafka partition key (typically equipment ID) to ensure ordering guarantees

5.2 Topic Mapping Strategy

The bridge maps the hierarchical, granular MQTT namespace to a flat, coarse Kafka namespace:

MQTT Topics (Source)	Kafka Topic (Target)
hollister/+/+/+/+/state	equipment_state
hollister/+/+/+/+/production/cycle_complete	production_events
hollister/+/+/+/+/production/count, .../reject	production_events
hollister/+/+/+/+/quality/#	quality_events
hollister/+/+/+/+/alarms/#	alarm_events
hollister/+/+/+/+/telemetry/# (filtered)	sensor_readings (sampled)
hollister/+/+/+/+/maintenance/#	maintenance_events

5.3 Bridge Implementation Options

Option	Strengths	Best For
Kafka Connect MQTT Source	Managed by Kafka ecosystem, declarative config, scales with Kafka	Standard integrations, minimal transformation
Confluent MQTT Proxy	Native MQTT interface to Kafka, no separate MQTT broker needed	Simpler architecture, Confluent customers
HiveMQ Kafka Extension	Integrated with HiveMQ broker, bidirectional support	HiveMQ deployments, bidirectional flows

Custom Stream Processor	Full control over transformation logic, complex enrichment	Advanced filtering, semantic enrichment, custom logic

5.4 Bridge Configuration Example

Example Kafka Connect MQTT Source configuration:

```
{
  "name": "mqtt-kafka-bridge-production",
  "config": {
    "connector.class": "io.confluent.connect.mqtt.MqttSourceConnector",
    "tasks.max": "4",

    "mqtt.server.uri": "ssl://mqtt-broker.hollister.com:8883",
    "mqtt.topics": "hollister/+/+/+/+/production/#",
    "mqtt.qos": "1",

    "kafka.topic": "production_events",

    "transforms": "extractFields,addTimestamp,setKey",
    "transforms.extractFields.type":
"org.apache.kafka.connect.transforms.ExtractField$Value",
    "transforms.extractFields.field": "payload",
    "transforms.addTimestamp.type":
"org.apache.kafka.connect.transforms.InsertField$Value",
    "transforms.addTimestamp.timestamp.field": "kafka_timestamp",
    "transforms.setKey.type": "org.apache.kafka.connect.transforms.ValueToKey",
    "transforms.setKey.fields": "equipment_id",

    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "https://schema-
registry.hollister.com:8081",

    "key.converter": "org.apache.kafka.connect.storage.StringConverter"
  }
}
```

6. Enterprise Layer: Kafka for Business Systems

6.1 Why Kafka for Enterprise

Enterprise systems—ERP, CRM, PLM, Supply Chain—care about the state and history of data rather than individual sensor blips. They need:

- Durability: Events must persist for days/months for audit, replay, and recovery
- Exactly-Once Semantics: Financial transactions cannot be duplicated or lost
- Consumer Independence: Each system consumes at its own pace without affecting others
- Replay Capability: Reprocess historical data when business logic changes
- Integration Patterns: Event sourcing, CQRS, saga patterns for distributed transactions

6.2 Kafka Topics for Manufacturing

Topic	Partitions	Retention	Consumers
equipment_state	By site (5)	7 days	ERP (availability), Analytics (OEE)
production_events	By equipment (50)	90 days	ERP (inventory), CRM (order status), PLM (actuals vs design)
quality_events	By product (20)	2 years	QMS, Regulatory (DHR), Analytics (predictive quality)
inventory_transactions	By warehouse (10)	1 year	ERP (stock levels), SCM (replenishment)
maintenance_events	By site (5)	1 year	CMMS, Analytics (predictive maintenance)
alarm_events	By severity (3)	90 days	Analytics (pattern detection), Compliance (safety logs)

6.3 Enterprise Consumer Patterns

6.3.1 SAP ERP Integration

When a "Job Finished" event arrives from MES via Kafka:

- Inventory Update: Consume from production_events, update material stock levels in SAP MM
- Cost Allocation: Post labor and machine hours to cost centers
- Production Confirmation: Update production order status, post goods receipt

```
// SAP Kafka Consumer - Production Event Handler
consumer.subscribe(["production_events"]);

for await (const message of consumer) {
  const event = ProductionEvent.deserialize(message.value);

  if (event.eventType === "BATCH_COMPLETE") {
    // Post goods receipt to SAP
    await sapClient.postGoodsReceipt({
      material: event.materialNumber,
      plant: event.site,
      quantity: event.quantityProduced,
      batch: event.batchNumber,
      storageLocation: event.destinationBin
    });

    // Confirm production order
    await sapClient.confirmProductionOrder({
      orderNumber: event.workOrderId,
      confirmationQuantity: event.quantityProduced,
      scrapQuantity: event.quantityRejected,
      activityType: "PROD",
      laborHours: event.laborMinutes / 60
    });
  }

  await consumer.commitOffsets();
}
```

6.3.2 CRM Integration (Order Status)

Customer-facing order status updates flow from production events:

- Order Tracking: When batch completes, update customer order status
- Ship Notification: Trigger shipment notification to customer
- Patient Portal: Update Secure Start™ portal with order progress

6.3.3 PLM Integration (Design vs. Actuals)

Product Lifecycle Management uses production data to improve designs:

- Process Capability: Compare actual process parameters to design specifications
- Quality Feedback: Feed defect data back to design for root cause analysis
- Continuous Improvement: Track yield trends against design revisions

6.3.4 Supply Chain Integration

Supply chain systems use production and inventory events for planning:

- Demand Signal: Production consumption triggers supplier replenishment
- Capacity Planning: Equipment availability feeds supply/demand balancing
- Supplier Scorecards: Material quality events update supplier performance metrics

7. End-to-End Data Flow

7.1 Complete Flow Example

This example traces a temperature reading from sensor to ERP:

Step	Component	Action
1	PLC/Sensor	Temperature sensor reads 185°C on injection molding barrel
2	Edge Gateway	OPC-UA server polls PLC, converts to MQTT message
3	MQTT Broker	Receives publish to hollister/BAL/MH1/LINE01/MOLD01/telemetry/temperature
4a	MES (MQTT Sub)	Immediately receives temperature, updates Digital Twin visualization
4b	Edge Analytics	Calculates rolling average, detects 185°C exceeds 180°C limit, publishes alarm
5	Integration Bridge	Subscribes to telemetry topic, applies deadband filter (change > 2°C), batches for Kafka
6	Kafka	Bridge publishes to sensor_readings topic with Avro schema, partitioned by equipment_id
7	Analytics Platform	Kafka consumer builds time-series for predictive model, detects anomaly pattern
8	Maintenance System	Receives predictive maintenance recommendation, creates work order in SAP PM

7.2 Production Cycle Complete Flow

This example traces a production cycle from completion to ERP booking:

```
Timeline of a Production Cycle Complete Event:

T+0ms     PLC detects cycle complete, increments part counter
T+5ms     Edge gateway reads counter via OPC-UA
T+10ms    MQTT publish: hollister/BAL/MH1/LINE01/MOLD01/production/cycle_complete
          Payload: {"count": 1, "cycle_time": 12.3, "material_lot": "LOT-2026-001"}

T+15ms    MES receives MQTT message (QoS 1)
          → Updates work order progress
          → Records cycle time in eDHR
          → Decrements material lot quantity

T+20ms    Digital Twin receives update
          → Increments visual counter
          → Updates cycle time gauge

T+100ms   Bridge receives MQTT message
          → Enriches with work order context from MES API
          → Transforms to Avro ProductionEvent schema
          → Publishes to Kafka production_events topic

T+150ms   Kafka persists message (replication factor 3)
          → Available for all consumers

T+500ms   ERP Kafka consumer processes message
          → Posts goods receipt to SAP MM
          → Updates inventory levels
          → Posts production confirmation

T+1000ms  CRM Kafka consumer processes message
          → Updates customer order status
          → Calculates estimated ship date

T+2000ms  Analytics consumer processes message
          → Updates OEE calculation
          → Feeds predictive quality model
```

8. Implementation Guidance

8.1 Deployment Architecture

Component	Deployment Location	Sizing Guidance
MQTT Broker	Per site (edge data center)	HiveMQ cluster (3 nodes), 100K concurrent connections
Edge Gateway	Per plant area	Kepware server with 50K tags per instance
Integration Bridge	Per site or centralized	Kafka Connect cluster (3 workers), 4 connectors
Kafka Cluster	Centralized (cloud or on-prem)	Confluent Cloud or 5-node cluster, 100K events/sec
Schema Registry	With Kafka cluster	3-node cluster for HA

8.2 Recommended Technology Stack

Layer	Recommended	Alternatives
MQTT Broker	HiveMQ Enterprise	EMQX Enterprise, Mosquitto (small scale), AWS IoT Core
Edge Gateway	Kepware KEPServerEX	Ignition Edge, Azure IoT Edge, EdgeX Foundry
Integration Bridge	Kafka Connect + MQTT Connector	HiveMQ Kafka Extension, Custom Flink/Spark Streaming
Event Streaming	Confluent Platform / Apache Kafka	Amazon MSK, Azure Event Hubs (Kafka API), Redpanda
Schema Management	Confluent Schema Registry	AWS Glue Schema Registry, Apicurio Registry

8.3 Migration Strategy

For brownfield deployments, follow this phased approach:

Phase 1: Shadow Mode (4 weeks)

- Deploy MQTT broker alongside existing systems
- Configure edge gateways to publish to MQTT (dual-write)
- Validate data flow without impacting production systems

Phase 2: Bridge Activation (4 weeks)

- Deploy Kafka cluster and integration bridge
- Enable bridge to consume MQTT and produce to Kafka
- Connect analytics consumers to Kafka (read-only)

Phase 3: Enterprise Integration (8 weeks)

- Connect ERP/CRM Kafka consumers
- Migrate from legacy point-to-point integrations
- Decommission legacy data paths

Phase 4: Edge Optimization (Ongoing)

- Tune MQTT topic structure based on usage patterns

- Optimize bridge filtering and batching
- Add edge analytics capabilities

9. Best Practices and Anti-Patterns

9.1 Best Practices

Practice	Rationale
Use MQTT for edge, Kafka for enterprise	Each protocol optimized for its domain; don't force-fit
Filter at the edge	Reduce bandwidth and Kafka load by filtering noise before the bridge
Aggregate MQTT topics to Kafka	Kafka performs best with fewer, higher-throughput topics
Use Avro schemas in Kafka	Schema evolution, validation, and compact encoding for enterprise
Partition by equipment ID	Ensures ordering per equipment, enables parallel processing
Implement idempotent consumers	Handle at-least-once delivery; use event IDs to detect duplicates
Monitor bridge lag	Bridge backlog indicates capacity issues; alert before data loss
Test failover scenarios	Validate MQTT broker failover, Kafka consumer rebalancing, bridge recovery

9.2 Anti-Patterns to Avoid

Anti-Pattern	Why It's Problematic
Kafka at the edge	Kafka clients are heavy, require stable networks, and topic count doesn't scale to millions
MQTT for enterprise integration	No persistence, no replay, limited consumer group support for enterprise patterns
1:1 MQTT-to-Kafka topic mapping	Creates topic explosion in Kafka; 1M MQTT topics should map to ~10 Kafka topics
No bridge filtering	Flooding Kafka with every sensor reading wastes storage and overwhelms consumers
JSON everywhere	JSON is fine for MQTT; use Avro in Kafka for schema enforcement and efficiency
Ignoring consumer lag	Slow consumers can fall behind retention; data is lost silently
Synchronous enterprise calls from edge	Edge devices should publish events, not call ERP APIs directly

Appendix A: Glossary

Term	Definition
MQTT	Message Queuing Telemetry Transport – lightweight publish-subscribe messaging protocol designed for IoT and constrained devices
Kafka	Apache Kafka – distributed event streaming platform for high-throughput, durable, real-time data pipelines
QoS	Quality of Service – MQTT delivery guarantee levels (0=at most once, 1=at least once, 2=exactly once)
Topic	Named channel for message categorization (hierarchical in MQTT, flat in Kafka)
Partition	Kafka concept – ordered, immutable sequence of messages within a topic, enables parallelism
Consumer Group	Kafka concept – set of consumers that cooperatively process partitions, enabling load balancing
Offset	Position of a message within a Kafka partition, used for replay and consumer tracking
Retained Message	MQTT feature – broker stores last message per topic for new subscribers
Bridge	Component that transfers data between MQTT and Kafka, performing transformation and filtering
Avro	Apache Avro – compact binary serialization format with schema evolution support, commonly used with Kafka
Schema Registry	Service that stores and validates message schemas, ensuring producer-consumer compatibility
EDA	Event-Driven Architecture – design pattern where systems communicate via events rather than direct calls

End of Event-Driven Architecture Guide